

# 1. Arquitectura del Sistema — DocumentIA MVP

Proyecto: AI DocClassExt — SAREB

## 1.1 Descripción General

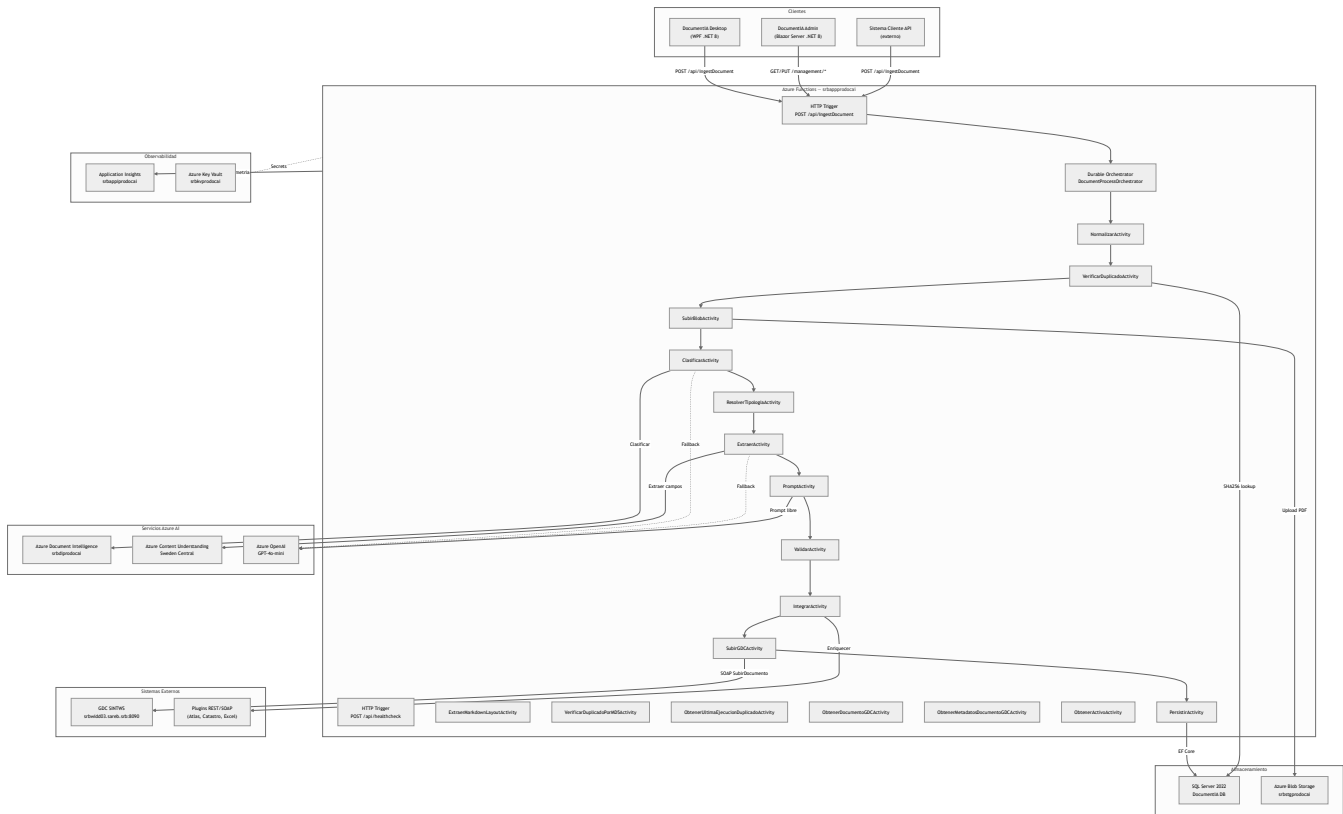
DocumentIA es un sistema de clasificación y extracción automatizada de documentos para SAREB. Procesa documentos inmobiliarios (notas simples registrales, tasaciones, resúmenes documentales) empleando servicios de IA de Azure para:

1. **Clasificar** el tipo documental mediante Azure AI Document Intelligence (modelos custom entrenados) con fallback a Azure OpenAI GPT-4o-mini (two-phase hierarchical: TDN1 family → TDN2 enriquecido con metadatos de BD).
2. **Extraer** campos estructurados mediante Azure Content Understanding o Document Intelligence, con fallback GPT.
3. **Validar** los datos extraídos contra un motor de reglas configurable por tipología (NIF, fechas, rangos, referencias catastrales, campos obligatorios).
4. **Enriquecer** los datos mediante un sistema de plugins de integración (REST, SOAP, DLL custom).
5. **Archivar** el documento en el Gestor Documental Corporativo (GDC SINTWS) via SOAP.
6. **Persistir** resultados, auditoría y métricas en SQL Server.

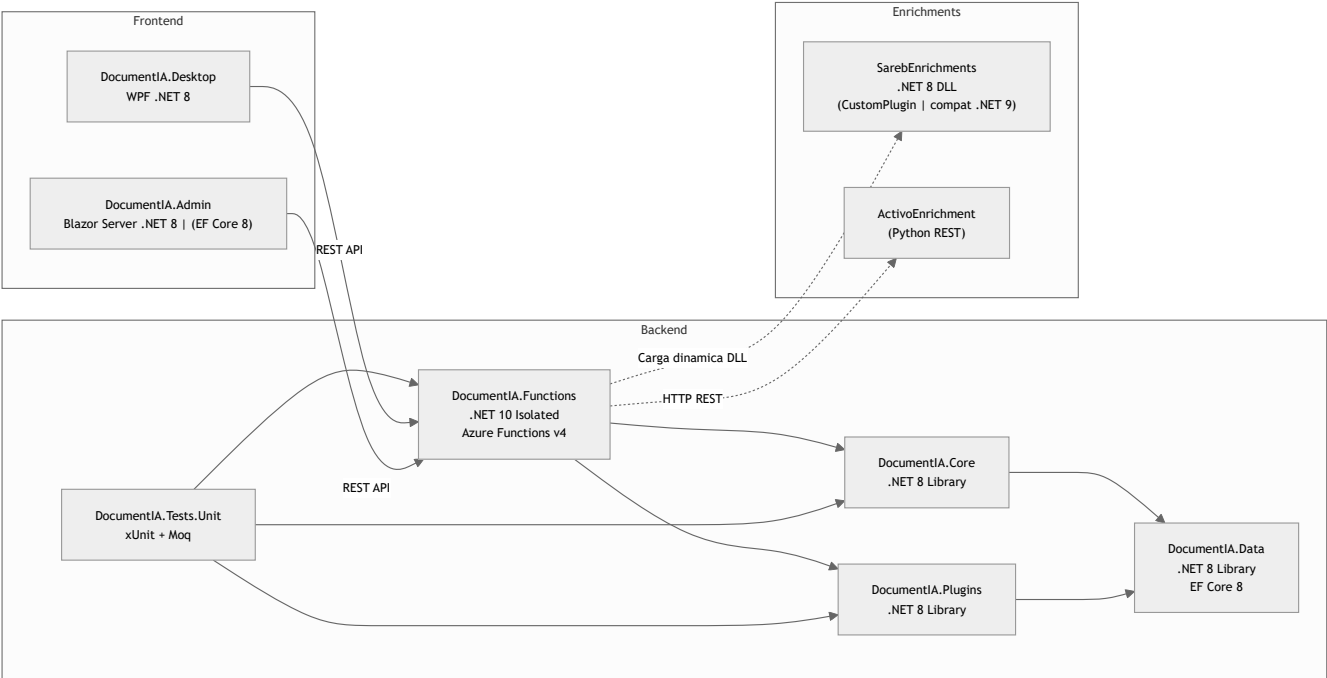
### Stack tecnológico

| Capa                    | Tecnología                                             |
|-------------------------|--------------------------------------------------------|
| Runtime                 | .NET 10 Isolated Worker                                |
| Orquestación            | Azure Durable Functions v4 (soportado hasta 2027)      |
| Clasificación IA        | Azure AI Document Intelligence (custom classifier)     |
| Extracción IA           | Azure Content Understanding + Azure DI (custom models) |
| LLM (fallback + prompt) | Azure OpenAI GPT-4o-mini                               |
| Base de datos           | SQL Server 2022 (EF Core 8 Code-First)                 |
| Almacenamiento blob     | Azure Blob Storage (Azurite en local)                  |
| Gestor documental       | GDC SINTWS (SOAP, srbwidd03.sareb.srb:8090)            |
| Frontend operativo      | WPF .NET 8 (MVVM, RestSharp)                           |
| Frontend Admin          | Blazor Server .NET 8                                   |
| CI/CD                   | Azure DevOps Pipelines (self-hosted agent)             |
| Observabilidad          | Application Insights + Log Analytics                   |

## 1.2 Diagrama de Arquitectura de Alto Nivel



### 1.3 Diagrama de Componentes de la Solucion .NET



### Responsabilidades por proyecto

| Proyecto              | Responsabilidad                                                                                                                                                          |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DocumentIA.Functions  | Triggers HTTP ( IngestDocument , healthcheck ), Orchestrator Durable, <b>17 Activities</b> , Providers de IA (DI, CU, GPT), servicios GDC y AssetResolver, autenticacion |
| DocumentIA.Core       | Modelos de dominio (ContratoEntrada/Salida), motor de validacion (11 reglas), configuracion de tipologias, interfaces de servicio                                        |
| DocumentIA.Data       | DbContext EF Core, 9 entidades, repositorios, migraciones, seed data                                                                                                     |
| DocumentIA.Plugins    | Infraestructura de plugins: IntegrationPlugin, PluginFactory, PluginManager, ResilientPlugin, RestPlugin, SoapPlugin, CustomPlugin                                       |
| DocumentIA.Tests.Unit | 33 clases de test (xUnit + Moq + FluentAssertions)                                                                                                                       |
| DocumentIA.Desktop    | WPF MVVM: ingesta de documentos, polling de estado en tiempo real, visualizacion de timeline                                                                             |
| DocumentIA.Admin      | Blazor Server: CRUD tipologias, gestion modelos AI, configuracion plugins                                                                                                |
| SarebEnrichments      | Plugin custom .NET: enriquecimiento de Nota Simple (cargas, riesgo, datos de activo)                                                                                     |

## 1.4 Principios y Patrones Arquitectonicos

### 1.4.1 Durable Functions — Orquestacion tipo Saga

El pipeline de procesamiento se implementa como un **Durable Function Orchestrator** ( DocumentProcessOrchestrator ) que coordina **17 activities** (no todas se ejecutan en cada peticion: hay early exits por duplicado, fallos GDC, skip de pasos opcionales como Prompt/AssetResolver). Las ventajas:

- **Durabilidad:** el estado de la orquestacion persiste automaticamente en Azure Storage. Si el host se reinicia, el orquestador resume donde quedo.
- **Replay-safe logging:** usa context.CreateReplaySafeLogger para evitar logs duplicados durante replays.
- **customStatus:** publica progreso en tiempo real que el frontend consume via polling del status endpoint de Durable.
- **Timeout patterns:** SubirGDCActivity usa Task.WhenAny con timer de 120s para evitar bloqueos indefinidos.

### 1.4.2 Strategy Pattern — Proveedores Configurables

Tanto clasificacion como extraccion usan el patron Strategy con un **router configurable**:

```
IClasificarDataProvider (interfaz)
├─ ConfigurableClasificarDataProvider (router)
│   ├── RuleBasedTdnClassifier ← reglas deterministas
│   ├── AzureDocumentIntelligenceClasificarProvider
│   ├── FoundryTdnRescueClassifier ← Foundry LLM fallback
│   ├── GptClasificarDataProvider (two-phase hierarchy) ← Phase 1: TDN1 family, Phase 2: TDN2 enriquecido
│   └─ MockClasificarDataProvider
```

**GptClasificarDataProvider:**

- **Phase 1:** Resuelve familia TDN1 (ej., ESCR) desde catálogo dinámico
- **Phase 2:** Resuelve tipología TDN2 enriquecida (ej., ESCR-06) desde catálogo filtrado + metadatos (tipologiaNombre, gptdescripcion) de BD
- **Caching:** 5 min TTL per-family via IMemoryCache
- **Enriquecimiento:** ConfiguracionJson contiene tipologiaNombre + gptdescripcion
- **Fallback:** Integrado en fallback chain tras DI y Foundry LLM
- **Documentación:** Ver ANEXO\_PROVIDER\_GPT\_CLASIFICACION\_DOS\_FASES.md

```

IExtraerDataProvider (interfaz)
├─ ConfigurableExtraerDataProvider (router)
│   ├─ AzureContentUnderstandingProvider      ← provider = "azure-content-understanding"
│   ├─ AzureDocumentIntelligenceExtraerDataProvider ← provider = "azure-document-intelligence"
│   ├─ GptDirectExtraerDataProvider          ← provider = "azure-openai" | "gpt" | "openai"
│   ├─ GptFallbackExtraerDataProvider        ← fallback automático cuando CU es insuficiente
│   └─ MockExtraerDataProvider              ← provider = "mock"

ILayoutMarkdownDataProvider (interfaz)
└─ AzureDocumentIntelligenceLayoutMarkdownProvider ← provider = "azure-document-intelligence-layout"

```

El proveedor se resuelve por prioridad: instrucciones de la petición → campo `extraction.provider` de la tipología → `Extraction.DefaultProvider` de configuración global. `GptDirectExtraerDataProvider` extrae directamente con OpenAI (sin CU previo), validando upfront que el modelo tiene endpoint, deployment y API key configurados.

Para `AzureContentUnderstandingProvider` está activo un patrón de resiliencia específico:

- Selección de modelo por tipología con `modelKey` + `secondaryModelKey` (round-robin cuando existe réplica).
- Timeout duro por intento ( `HardTimeoutSeconds` ) con evento de telemetría `CU.HardTimeout` .
- Circuit breaker por `ModelKey` con failover al modelo alternativo ( `CU.CircuitOpen` , `CU.CircuitFailover` , `CU.CircuitClosed` , `CU.CircuitRejected` ).
- Retorno de metadatos efectivos de ejecución ( `ModelKeyEfectivo` , `EndpointEfectivo` , `ProcessingLocationEfectiva` ) para trazabilidad.

### 1.4.2a Patrón RegistryLoader — Configuración de Proveedores IA desde BD

Todos los parámetros de conexión de los proveedores IA (**endpoint**, **apiKey**, **authMode**, **apiVersion**, etc.) se almacenan **exclusivamente en la tabla `ModeloConfigs` de BD**. No hay ninguna clave de configuración en `appsettings` ni en `local.settings.json` para estos parámetros.

Cada tipo de modelo tiene su propio loader que hereda de `ModelRegistryLoader<TConfig>` :

| Loader                                         | Tipo modelo                           | Configuración cargada                  |
|------------------------------------------------|---------------------------------------|----------------------------------------|
| <code>ClassificationModelRegistryLoader</code> | <code>TipoModelo.Clasificacion</code> | <code>ClassificationModelConfig</code> |
| <code>ExtractionModelRegistryLoader</code>     | <code>TipoModelo.Extraccion</code>    | <code>ExtractionModelConfig</code>     |
| <code>PromptModelRegistryLoader</code>         | <code>TipoModelo.Prompt</code>        | <code>PromptModelConfig</code>         |
| <code>LayoutModelRegistryLoader</code>         | <code>TipoModelo.Layout</code>        | <code>LayoutModelConfig</code>         |

Los proveedores reciben el loader por inyección de dependencias y obtienen el modelo con `GetModel(key)` o `GetDefaultModel()` . Los archivos `config/{tipo}/models.json` solo actúan como seed/bootstrap de un entorno sin datos o como referencia histórica; la configuración válida en runtime es la publicada en BD y puede diferir de esos JSON.

### 1.4.3 Repository Pattern

Acceso a datos mediante repositorios inyectados ( `IDocumentoRepository` , `ITipologiaRepository` , `IAuditoriaRepository` , etc.) sobre EF Core DbContext.

### 1.4.4 Plugin Architecture — Carga Dinámica

Los plugins de integración siguen una arquitectura extensible:

- **IIIntegrationPlugin**: interfaz común con `InitializeAsync` , `ExecuteAsync` , `HealthCheckAsync` .
- **PluginFactory**: crea instancias según configuración ( `REST` → `RestPlugin` , `SOAP` → `SoapPlugin` , `Custom` → `CustomPlugin` ).
- **PluginManager**: registro y lookup de plugins por clave.
- **CustomPlugin**: carga DLLs externas desde `AppContext.BaseDirectory/plugins/` usando reflexión.
- **Configuración de plugins por tipología**: se lee desde la tabla `PluginTipologiaConfigs` en BD. El fichero `config/tipologias/{codigo}.plugins.json` queda limitado a seed inicial, plantilla o referencia histórica.

Regla general: tipologias, modelos IA y plugins por tipologia tienen como fuente de verdad la BBDD gestionada por Admin API/DocumentIA.Admin. Los JSON fisicos no deben editarse como mecanismo de cambio productivo ni borrarse sin confirmacion explicita.

### 1.4.5 Circuit Breaker + Retry (Decorator Pattern)

La resiliencia se implementa mediante decoradores:

- **ResilientPlugin:** envuelve cualquier `IIntegrationPlugin` con retry exponencial y circuit breaker.
  - Retry: `MaxRetries` intentos con backoff `InitialDelayMs * 2^(attempt-1)`.
  - Circuit breaker: abre tras 5 fallos consecutivos, se resetea tras 5 minutos (half-open trial).
- **ResilientGdcService:** envuelve `GdcService` (SOAP) con la misma logica.
  - Configurable via `GDC:MaxRetries`, `GDC:CircuitBreakerFailures`, `GDC:CircuitBreakerDurationMs`.

### 1.4.6 Dual Authentication Mode

Los proveedores de IA soportan dos modos de autenticacion ( `AuthMode` ):

| Modo                                | Mecanismo                                                                             | Uso actual                           |
|-------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------|
| <code>ApiKey</code>                 | Header <code>Ocp-Apim-Subscription-Key</code> (DI) / <code>api-key</code> (OpenAI/CU) | Produccion (actual)                  |
| <code>DefaultAzureCredential</code> | Bearer token via Managed Identity                                                     | Preparado, pendiente asignacion RBAC |

`DocumentIntelligenceAuthHelper` implementa la logica con lazy singleton de `DefaultAzureCredential` thread-safe.

## 1.5 Decisiones Arquitectonicas (ADR)

### ADR-001: Azure Durable Functions como motor de orquestacion

| Aspecto        | Detalle                                                                                                                                                                                                                                                                                                                                                                               |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto       | Se necesita orquestar un pipeline de 8-13 pasos asincrono con llamadas a servicios externos de IA (latencia 5-60s por paso), gestion de estados, retry y monitorizacion en tiempo real.                                                                                                                                                                                               |
| Opciones       | (A) Azure Durable Functions, (B) Azure Logic Apps, (C) Colas + funciones independientes, (D) Microservicios con Dapr                                                                                                                                                                                                                                                                  |
| Decision       | <b>(A) Azure Durable Functions</b>                                                                                                                                                                                                                                                                                                                                                    |
| Justificaci on | <ul style="list-style-type: none"><li>- Modelo de codigo C# nativo (no YAML/JSON designer).</li><li>- Persistencia automatica de estado en Storage.</li><li>- <code>customStatus</code> para polling de progreso.</li><li>- Retry policies y timer support built-in.</li><li>- Consumption plan (pago por ejecucion).</li><li>- Todo en un unico proyecto .NET desplegable.</li></ul> |
| Riesgos        | Replay semantics requieren cuidado (no I/O en el orchestrator). Tamaño de input/output limitado por Storage.                                                                                                                                                                                                                                                                          |

### ADR-002: EF Core Code-First sobre SQL Server

| Aspecto       | Detalle                                                                                                                                                                                                                                                                                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto      | Se necesita persistir documentos, resultados, auditoria y configuracion con schema evolutivo durante el MVP.                                                                                                                                                                                                                                                             |
| Opciones      | (A) EF Core Code-First, (B) Database-First, (C) Dapper raw SQL                                                                                                                                                                                                                                                                                                           |
| Decision      | <b>(A) EF Core Code-First</b>                                                                                                                                                                                                                                                                                                                                            |
| Justificacion | <ul style="list-style-type: none"><li>- Migraciones automaticas aplican cambios de schema sin scripts manuales.</li><li>- <code>DbContext.Database.Migrate()</code> en startup para dev local.</li><li>- Seed data desde archivos JSON de config.</li><li>- Facilidad de evolucionar el modelo durante MVP.</li><li>- Repository pattern para desacoplamiento.</li></ul> |
| Trade-offs    | Menos control sobre SQL generado. Para consultas criticas de rendimiento futuras se puede usar raw SQL/Dapper puntualmente.                                                                                                                                                                                                                                              |

ADR-003: Azure DI + GPT fallback para clasificacion

| Aspecto       | Detalle                                                                                                                                                                                                                                                                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto      | La clasificacion requiere alta precision (>85%) pero los modelos custom de DI pueden tener baja confianza en documentos atipicos.                                                                                                                                                                                                                       |
| Opciones      | (A) Solo Azure DI, (B) Solo GPT-4o-mini, (C) DI + GPT fallback                                                                                                                                                                                                                                                                                          |
| Decision      | <b>(C) DI con fallback a GPT</b>                                                                                                                                                                                                                                                                                                                        |
| Justificacion | <ul style="list-style-type: none"><li>- DI es rapido y barato para documentos que coinciden con el entrenamiento.</li><li>- GPT maneja documentos fuera de distribucion con reasoning.</li><li>- Umbral configurable por tipologia ( <code>clasifUmbralFallback</code> ).</li><li>- Mismo patron para extraccion: CU primario + GPT fallback.</li></ul> |
| Metricas      | DI: ~2-5s, GPT: ~5-15s. Fallback solo se activa cuando confianza DI < umbral.                                                                                                                                                                                                                                                                           |

ADR-004: Sistema de plugins DLL dinamico

| Aspecto           | Detalle                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto          | Cada tipologia puede requerir enriquecimiento diferente (REST a catastro, SOAP a atlas, logica custom). Debe ser extensible sin modificar el core.                                                                                                                                                                                                                                    |
| Opciones          | (A) Plugins como DLLs cargadas dinamicamente, (B) Microservicios por plugin, (C) Azure Logic Apps por plugin                                                                                                                                                                                                                                                                          |
| Decision          | <b>(A) Plugins DLL + REST/SOAP</b>                                                                                                                                                                                                                                                                                                                                                    |
| Justificacio<br>n | <ul style="list-style-type: none"><li>- DLLs compiladas externamente (SarebEnrichments.dll), cargadas desde <code>AppContext.BaseDirectory/plugins/</code> .</li><li>- Plugins REST/SOAP no requieren codigo .NET.</li><li>- PluginFactory + ResilientPlugin abstraen la complejidad.</li><li>- Prioridad configurable: plugins criticos (P1) detienen la cadena si fallan.</li></ul> |

ADR-005: Dual AuthMode (ApiKey + Managed Identity)

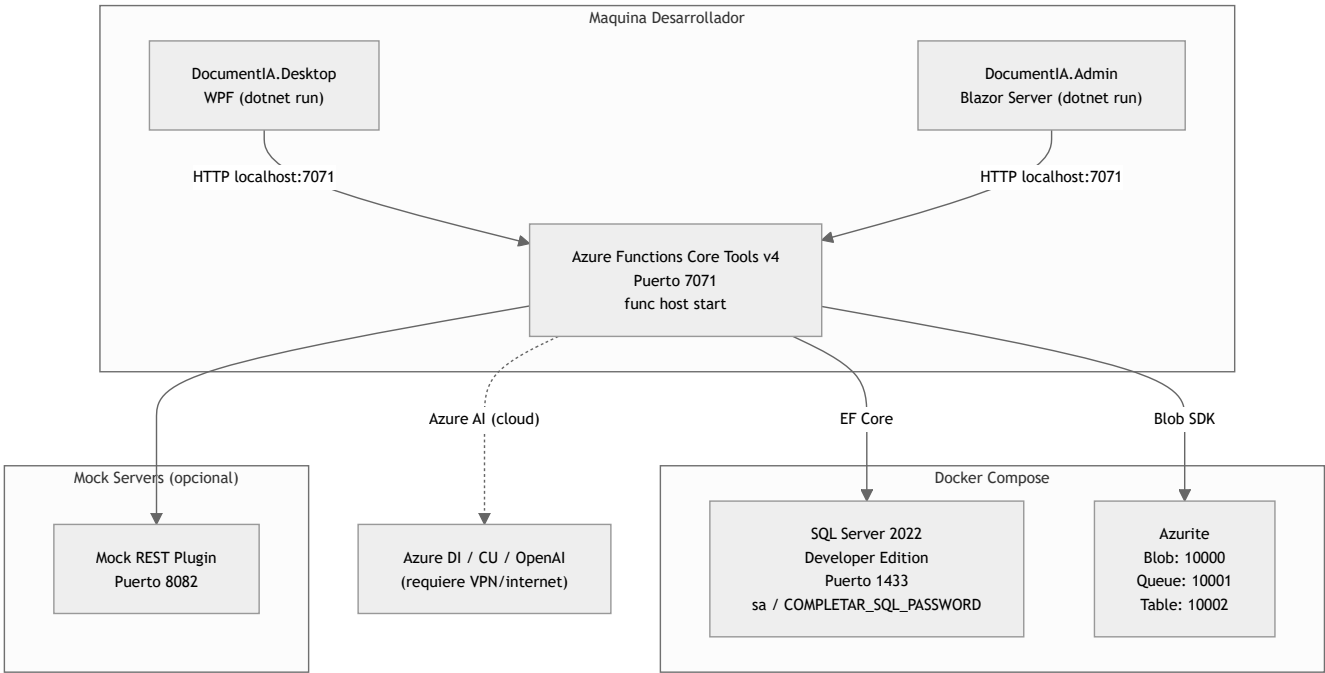
| Aspect<br>o  | Detalle                                                                                                                                                                                                         |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contex<br>to | El MVP necesita funcionar rapidamente con API Keys, pero la produccion final debe usar Managed Identity (zero-secret).                                                                                          |
| Decisio<br>n | Implementar ambos modos desde el inicio, seleccionables por configuracion ( <code>AuthMode: "ApiKey"</code> o <code>"DefaultAzureCredential"</code> ).                                                          |
| Estado       | ApiKey activo en produccion. MI preparado en codigo, pendiente asignacion roles RBAC ( <code>Cognitive Services User</code> ) a la System Managed Identity <code>&lt;MANAGED_IDENTITY_PRINCIPAL_ID&gt;</code> . |

ADR-006: Docker SQL local con auto-migraciones

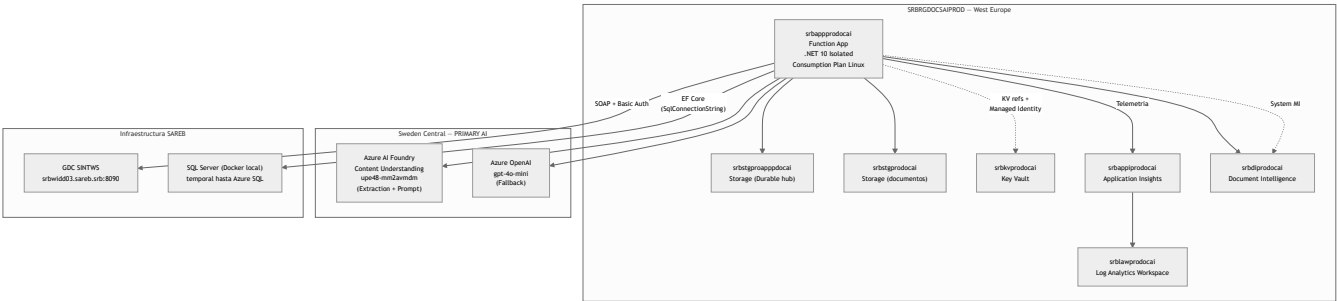
| Aspecto    | Detalle                                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contexto   | El desarrollo local necesita SQL Server sin depender de Azure SQL (coste + conectividad).                                                                                                            |
| Decision   | <code>docker-compose.yml</code> con SQL Server 2022 Developer + Azurite. <code>Program.cs</code> aplica migraciones automaticamente en startup ( <code>RunDatabaseMigrationsOnStartup=true</code> ). |
| Transicion | Cuando Azure SQL ( <code>srbsqlprodcaí</code> ) este disponible, solo cambia <code>SqlConnectionString</code> . El seed se aplica desde <code>config/</code> automaticamente.                        |

## 1.6 Diagrama de Despliegue

### 1.6.1 Entorno Local (Desarrollo)

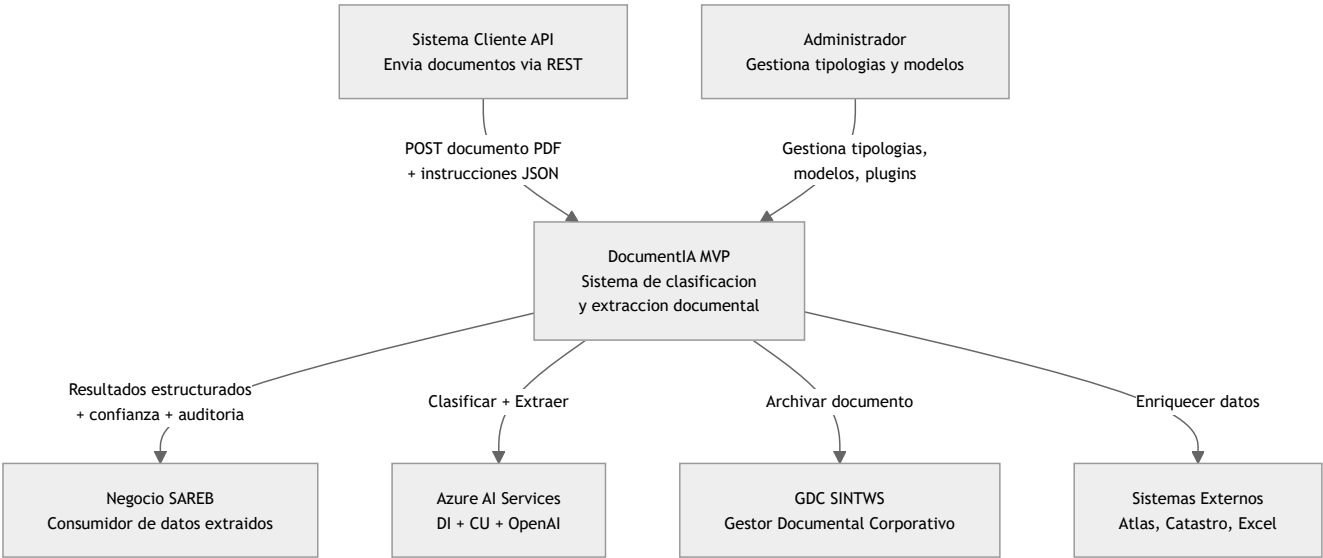


### 1.6.2 Entorno Azure (Produccion)

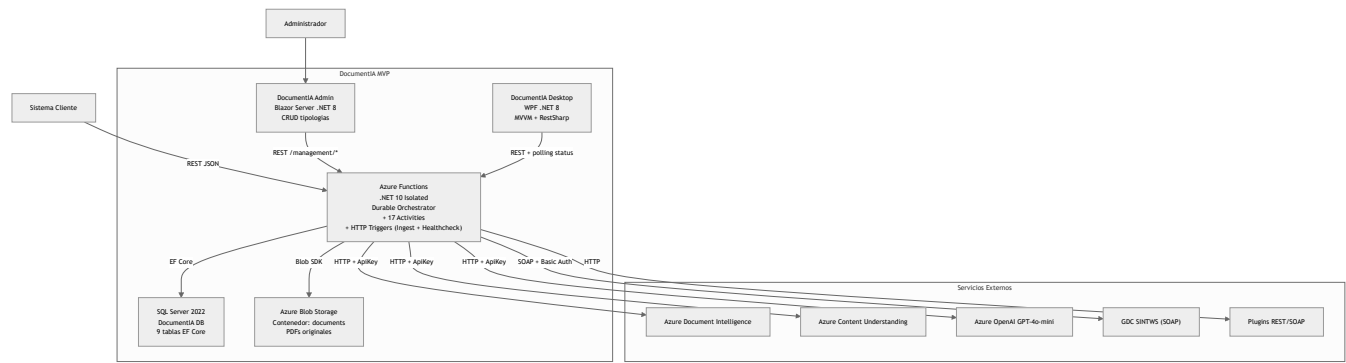


## 1.7 Diagramas de Contexto C4

### 1.7.1 C4 Level 1 — Contexto del Sistema



1.7.2 C4 Level 2 — Contenedores



1.8 Escalabilidad, Disponibilidad y Resiliencia

Escalabilidad

| Aspecto       | Estrategia                                                                                                                   |
|---------------|------------------------------------------------------------------------------------------------------------------------------|
| Compute       | Azure Functions Consumption Plan: escala automatica de 0 a N instancias segun carga. Sin gestion de servidores.              |
| Concurrencia  | Durable Functions gestiona multiples orquestaciones en paralelo. Cada instancia es independiente.                            |
| Storage       | Azure Blob Storage: escalado nativo sin limites practicos. Durable task hub en Storage separado ( srbstgproappdpdocal ).     |
| Base de datos | SQL Server 2022 (actual: Docker local). Plan: Azure SQL Elastic Pool para escalado vertical/horizontal.                      |
| AI Services   | Azure AI tiene rate limits por region. Content Understanding en Sweden Central (alta capacidad). DI y OpenAI en West Europe. |

Disponibilidad



| Aspecto      | Estrategia                                                                                                 |
|--------------|------------------------------------------------------------------------------------------------------------|
| Function App | Consumption plan con SLA 99.95%. Health checks integrados.                                                 |
| Durabilidad  | Durable Functions persiste estado en Storage. Tolerante a reinicios del host.                              |
| Idempotencia | Deduplicacion por SHA256: un documento identico no se reprocesa (salvo <code>forceReprocess=true</code> ). |
| GDC          | Consulta previa ( <code>ConsultarDocumentoAsync</code> ) antes de subir. Si ya existe, no duplica.         |

Resiliencia

| Componente             | Mecanismo                                                                                                                                                      |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Plugins                | <code>ResilientPlugin</code> : retry exponencial ( <code>InitialDelayMs * 2^attempt</code> ) + circuit breaker (5 fallos → open, 5 min → half-open)            |
| GDC SOAP               | <code>ResilientGdcService</code> : retry configurable ( <code>GDC:MaxRetries</code> ) + circuit breaker ( <code>GDC:CircuitBreakerFailures/DurationMs</code> ) |
| SubirGDC timeout       | <code>Task.WhenAny</code> con timer de 120s en el orchestrator. Si timeout, marca "Timeout" y continua a persistencia.                                         |
| Clasificacion fallback | Si confianza DI < umbral, automaticamente se invoca GPT-4o-mini.                                                                                               |
| Extraccion fallback    | Si completitud CU < umbral, se redirige a GPT para campos faltantes.                                                                                           |
| Plugin critico         | Plugin con <code>Priority=1</code> falla → se detiene la cadena de plugins. Plugins no criticos: log warning y continua.                                       |
| Error general          | Try/catch global en el orchestrator. Estado final <code>ERROR</code> con mensaje. Siempre persiste lo que tenga hasta ese punto.                               |

Limites conocidos

| Limite                         | Valor                                                 | Mitigacion                                                                                                     |
|--------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Tamaño input Durable Functions | ~60 KB (Storage entity)                               | El base64 del documento se pasa en el input; documentos muy grandes pueden requerir blob-reference pattern     |
| Timeout Activity               | Sin limite explicito (excepto GDC: 120s)              | Posibilidad de definir timeouts por activity                                                                   |
| Rate limit Azure DI            | Varía por tier (S0: 15 TPS)                           | Durable Functions serializa por instancia; N instancias paralelas podrian saturar                              |
| SSL GDC                        | Certificado CA corporativo SAREB no confiado en Linux | <code>GDC:BypassSslValidation=true</code> (solo para host Linux; en Windows la CA se instala en el cert store) |

1.9 Referencias a Documentacion Relacionada

| Documento                                                  | Contenido                                                      |
|------------------------------------------------------------|----------------------------------------------------------------|
| <a href="#">03_DISENO_TECNICO_DETALLADO.md</a>             | Problema de negocio, casos de uso, requisitos y diseno tecnico |
| <a href="#">03_DISENO_TECNICO_DETALLADO.md</a>             | Flujo pipeline, secuencias, modelo ER, contratos               |
| <a href="#">04_MANUAL_EXPLOTACION.md</a>                   | Instalacion, despliegue, operacion                             |
| <a href="#">CONTRATO_API_HTTP.md</a>                       | Contrato de API REST detallado                                 |
| <a href="#">MANUAL_PLUGINS.md</a>                          | Guia de desarrollo de plugins                                  |
| <a href="#">ESPECIFICACION_CAPA_SERVICIO_GDC_SINTWS.md</a> | Integracion con GDC SINTWS                                     |